United International University
Department of Computer Science and Engineering

CSE 3421: Software Engineering Mid: Spring 2026

Total Marks: 30 Time: 1 hour and 30 minutes

Any examinee found adopting unfair means will be expelled from the trimester / program as per UIU disciplinary rules.

Answer all five questions. The numbers on the right of the questions denote their marks.

1. (a) **Explain** why **staging** is required before the actual **commit** with an **example**. [CO2] (2)

   (b) **Write GIT commands** for the following **tasks**: [CO2] (4)

   I Set up the remote repository https://github.com/company/project.git within your local system.

   II Create 2 separate branches for managing tasks related to login and signup. Assume "login.html" and "signup.html" have been created within the separate branches, and commit the changes.

   III Merge the login branch with the signup branch, and save the changes to the remote repository.

   IV Sync the local main branch with the updated remote repository.

2. (a) What is the **difference** between **Functional Requirements** and **Non-Functional Requirements**? Explain with a suitable **example** for each. [CO1] (2)

   (b) You are working as a security analyst for a fintech mobile banking company. (4)

   1. **Case 1**: A user installs a "battery saver" app from a third-party website on their smartphone. The app works properly at first, but after a few days, the phone becomes slow, and unknown apps start running in the background without permission.
   2. **Case 2**: A user connects to a free public Wi-Fi network at a café to access their mobile banking app. An attacker secretly intercepts the communication between the user and the bank's server and modifies transaction details without the user's knowledge.

   What type of **cyber-attacks** are responsible for each case described above? Explain your answers with proper **reasoning**. [CO2]

3. (a) **Differentiate** between **Requirement Elicitation** and **Requirement Validation**. [CO1] (2)

   (b) A gaming startup is building a multiplayer mobile game where users can battle each other in real-time, customize avatars, and earn rewards. The team starts development with only a rough idea of gameplay mechanics. As development progresses, they frequently experiment with new features like power-ups, maps, and reward systems. After each release, players provide feedback, which often leads to major gameplay changes. The team also emphasizes high code quality, frequent testing, and regular integration to avoid bugs in live matches. Which software development **model** would be most **suitable** for this project? **Explain** with proper reasoning. [CO2] (4)

4. (a) **Differentiate** between **Product Backlog** and **Sprint Backlog**. [CO2] (2)

   (b) A small team is developing a community gardening app using **Scrum**. They work in fixed four-week iterations to maintain a steady rhythm of updates. Every morning, the team holds a strict 15-minute meeting to coordinate tasks for the next 24 hours. A working product output is delivered at the end of each iteration. The team follows a clear checklist or standard to determine when a product output is complete and of proper quality. After the iteration ends and features are delivered and presented to the stakeholders, the team conducts a reflection meeting. During this session, they identified that their code-testing process was too slow and decided to automate part of it in the next iteration to improve efficiency.
   Which specific Scrum **events** and **artifacts** are indicated in the scenario? Explain your answers with proper **reasoning**. [CO2] (4)

5. (a) You are reviewing the Customer class in a massive e-commerce codebase, and you notice it has grown too large. It has many fields, and there are several unrelated methods within the class (storing personal details, computing billing, and formatting mailing addresses). This class is a clear code-smell example. **Explain** how you would approach refactoring this class to improve its design, and **why** your approach makes the code easier to read and maintain. [CO2] (2)

   (b) **Refactor** the following code: [CO2] (4)

```
public class A {
    public void X(int n) {
        // factorial
        int f = 1;
        for (int i = 1; i <= n; i++) {
            f = f * i;
        }
        System.out.println("Factorial: " + f);

        // fibonacci sum
        int a = 0, b = 1, s = 0;
        for (int i = 0; i < n; i++) {
            s = s + a;
            int t = a + b;
            a = b;
            b = t;
        }
        System.out.println("Fibonacci sum: " + s);
    }
}
```